

Oracle复习

Oracle复习

安装

第1章Oracle 10g 数据库概述

ORACLE数据库的特点

ORACLE查询工具

第2章Oracle 10g体系结构及安全管理

ORACLE数据库物理存储结构

ORACLE数据库逻辑存储结构

ORACLE数据库用户的创建，修改，删除

用户权限的分配

第3章Oracle 10g空间管理

创建，修改，删除，查询表空间

SQL语言基础

索引的增删改

视图的创建和删除

同义词和序列

第4章Oracle高级查询、事务、过程及函数

SQL函数

多表查询

事务处理

理解过程及函数

第5章Oracle PL/SQL编程基础

PL/SQL

PL/SQL语言基础

PL/SQL异常处理

游标的使用

第6章Oracle PL/SQL高级特性

触发器的作用

触发器的创建

程序包的建立和使用

第7章Oracle备份与恢复

常用备份与恢复

第8章Oracle优化技术

SQL语句优化

安装

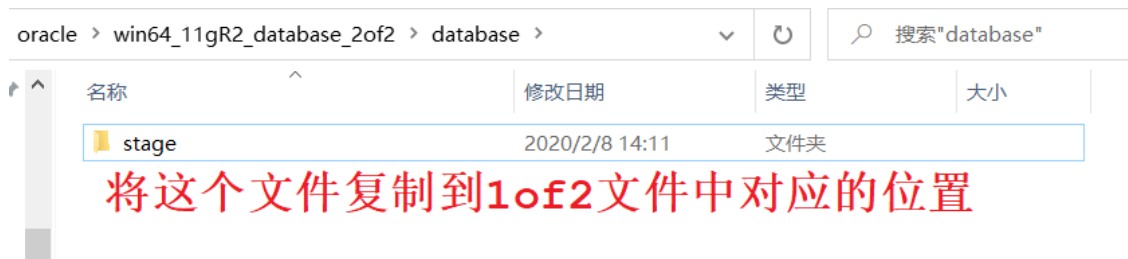
官网下载链接：<https://www.oracle.com/database/technologies/oracle-database-software-downloads.html>

(11.2.0.1.0)		
Name	Download	Note
Microsoft Windows (32-bit)	 File 1 ,  File 2 (2GB)	See All
Microsoft Windows (x64)	 File 1 ,  File 2 (2GB)	See All
Linux x86	 File 1 ,  File 2 (2GB)	See All

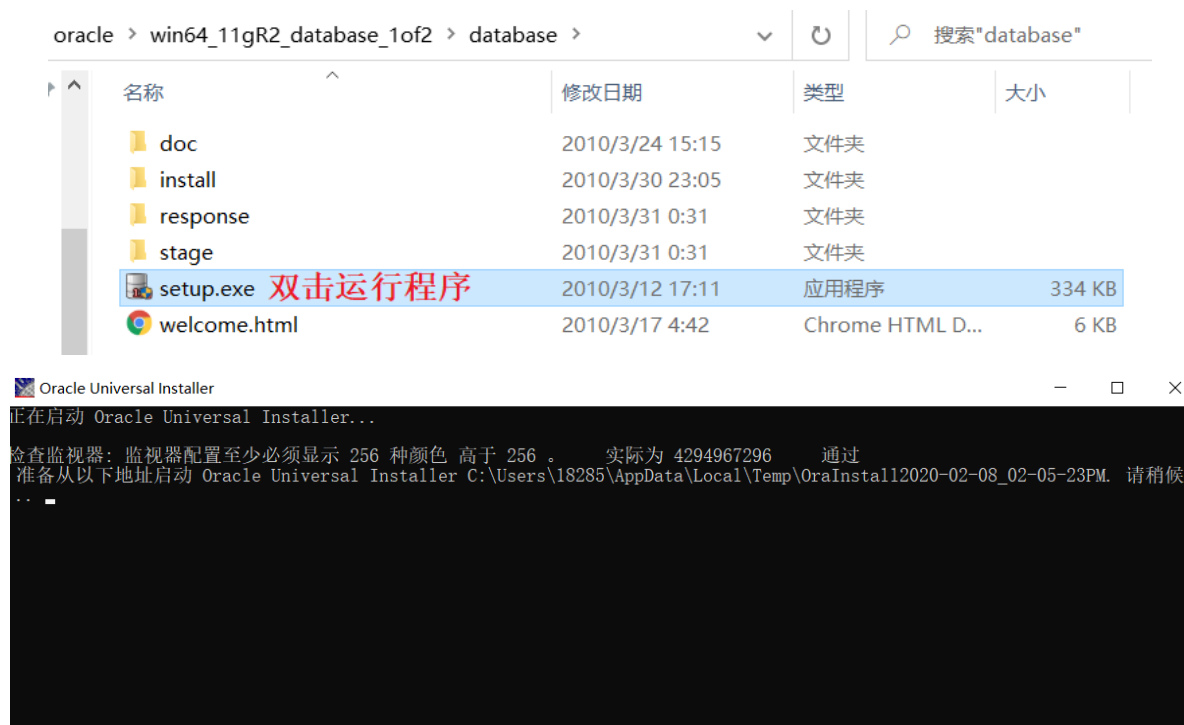
解压文件：

	win64_11gR2_database_1of2	2020/2/8 13:41
	win64_11gR2_database_2of2	2020/2/8 13:45
	win64_11gR2_database_1of2.zip	2020/2/8 1:32
	win64_11gR2_database_2of2.zip	2020/2/8 1:09

先整合两个文件的database：



运行安装程序：





解决ins-13001问题:

```

<OPERATING_SYSTEM RELEASE="6.0">
  <VERSION VALUE="3"/>
  <ARCHITECTURE VALUE="64-bit"/>
  <NAME VALUE="WindowsVista"/>
  <ENV_VAR_LIST>
    <ENV_VAR NAME="PATH" MAX_LENGTH="1023" />
  </ENV_VAR_LIST>
</OPERATING_SYSTEM>
<OPERATING_SYSTEM RELEASE="6.1">
  <VERSION VALUE="3"/>
  <ARCHITECTURE VALUE="64-bit"/>
  <NAME VALUE="Windows 7"/>
  <ENV_VAR_LIST>
    <ENV_VAR NAME="PATH" MAX_LENGTH="1023" />
  </ENV_VAR_LIST>
</OPERATING_SYSTEM>
<OPERATING_SYSTEM RELEASE="6.1">
  <VERSION VALUE="3"/>
  <ARCHITECTURE VALUE="64-bit"/>
  <NAME VALUE="WindowsServer2008R2"/>
  <ENV_VAR_LIST>
    <ENV_VAR NAME="PATH" MAX_LENGTH="1023" />
  </ENV_VAR_LIST>
</OPERATING_SYSTEM>
</IFIED_SYSTEMS>

```

修改为:

```
<OPERATING_SYSTEM RELEASE="6.2">
  <VERSION VALUE="3"/>
  <ARCHITECTURE VALUE="64-bit"/>
  <NAME VALUE="Windows 10"/>
  <ENV_VAR_LIST>
    <ENV_VAR NAME="PATH" MAX_LENGTH="1023" />
  </ENV_VAR_LIST>
</OPERATING_SYSTEM>
```

重新运行安装文件，接下来就可以继续安装了：

Oracle Database 11g 发行版 2 安装程序 - 安装数据库 - 步骤 1/9

配置安全更新

提供电子邮件地址以用于接收有关安全问题的通知, 安装该产品并启动配置管理器。 [查看详细资料\(V\)](#)。

电子邮件(M):

如果使用 My Oracle Support 电子邮件地址/用户名, 会更加方便。

☐ 我希望通过 My Oracle Support 接收安全更新(W)。

My Oracle Support 口令(O):

帮助(H) < 后退(B) 下一步(N) > 完成(F) 取消



- 配置安全更新 安装选项 - 网络安装选项 - 安装类型 - 典型安装 - 先决条件检查 - 概要 - 安装产品 - 完成	选择以下任意安装选项。 ☒ 创建和配置数据库(C) ☐ 仅安装数据库软件(I) ☐ 升级现有的数据库(U)
帮助(H) < 后退(B) 下一步(N) > 完成(F) 取消	



- 配置安全更新 安装选项 系统类 - 典型安装 - 先决条件检查 - 概要 - 安装产品 - 完成	☒ 桌面类(D) 如果要在笔记本或桌面类系统中安装, 则选择此选项。此选项包括启动数据库并允许采用最低配置。 ☐ 服务器类(S) 如果要在服务器类系统中进行安装 (如在生产数据中心内部署 Oracle 时使用的内容), 则选择此选项。此选项允许使用更多高级配置选项。
帮助(H) < 后退(B) 下一步(N) > 完成(F) 取消	

典型安装配置

配置安全更新

安装选项

系统类

典型安装

先决条件检查

概要

安装产品

完成

使用基本配置执行完全数据库安装。

不能有中文目录

Oracle 基目录(S):

D:\study-GZ\oracle_client

浏览(B)

软件位置(L):

D:\study-GZ\oracle_client\product\11.2.0\dbhome_1

浏览(Q)

数据库文件位置(O):

D:\study-GZ\oracle_client\oradata

浏览(V)

数据库版本(E):

企业版 (3.34GB)

字符集(I):

默认值 (ZHS16GBK)

全局数据库名(G):

orcl

管理口令(P):

确认口令(Q):

口令格式：必须有大、小写字母、数字组成

帮助(H)

< 后退(B)

下一步(N) >

完成(F)

取消

接下来一直点击下一步，有弹框选择：是 和 完成

安装产品

配置安全更新

安装选项

系统类

典型安装

先决条件检查

概要

安装产品

完成

进度

21%

将文件提取到 'D:\study-GZ\oracle_client\product\11.2.0\dbhome_1'。

状态

Oracle Database 安装	正在进行
• 准备	成功
• 复制文件	正在进行
• 安装程序文件	暂挂
Oracle Database 配置	暂挂

等待安装

详细资料(D)

重试(R)

跳过(S)

帮助(H)

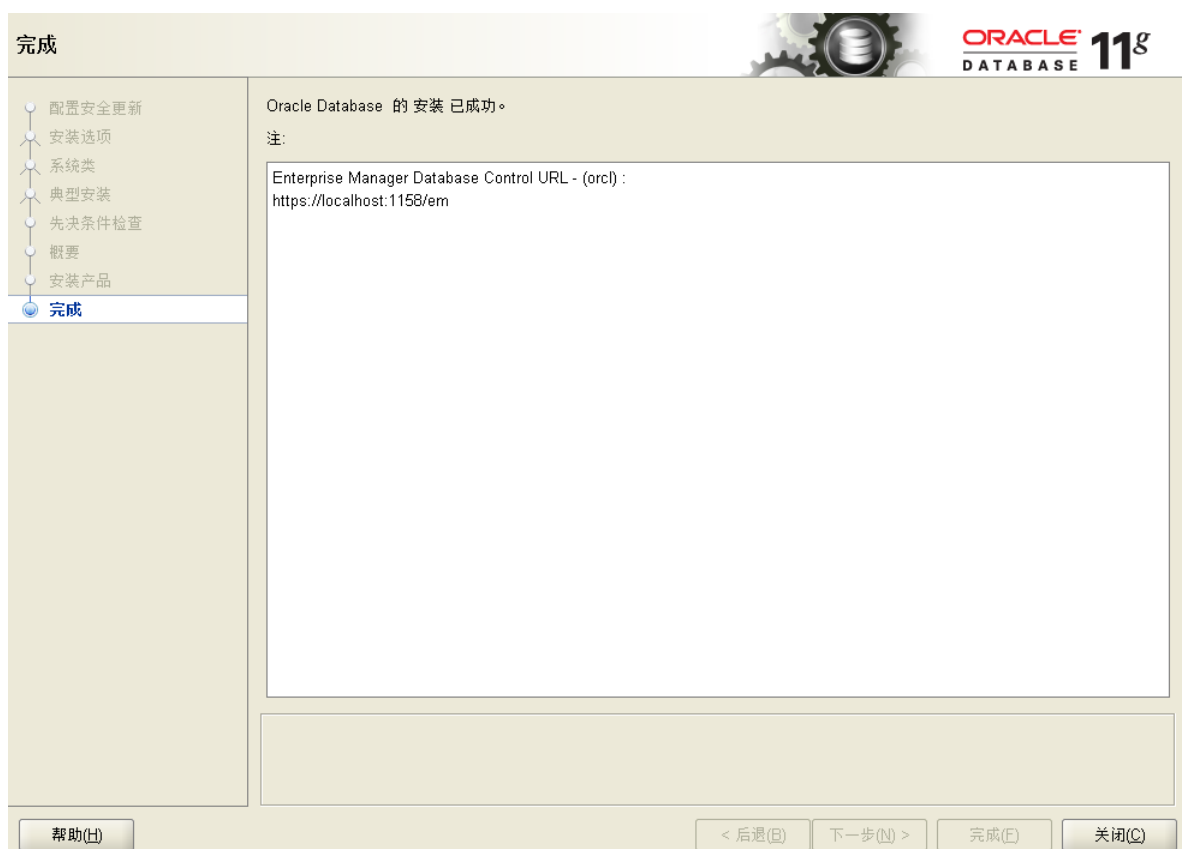
< 后退(B)

下一步(N) >

完成(F)

取消

等待安装：最后的弹出界面点击：确定



第1章Oracle 10g 数据库概述

ORACLE数据库的特点

Oracle数据库是一款业界首个为网络计算而设计的大型数据库

Oracle数据库受大、中型企业的青睐

Oracle数据库具有如下特点：

- 数据库高可用性
- 可伸缩性
- 安全性
- 可管理性
- 数据仓储
- 应用软件开发
- 集成性

ORACLE查询工具

--安装Oracle时自带安装的sqlplus.

-第三方可视化管理工具PL/SQL Developer

--Oracle公司自己的Oracle可视化管理工具SQL Developer

下载地址：<https://www.oracle.com/technetwork/developer-tools/sql-developer/downloads/index.html>

下载地址: <https://www.oracle.com/technetwork/developer-tools/sql-developer/downloads/index.html>

第2章Oracle 10g体系结构及安全管理

ORACLE数据库物理存储结构



ORACLE数据库逻辑存储结构



ORACLE数据库用户的创建，修改，删除

```
create user zhangsan identified by 123456; 创建用户

alter user zhangsan identified by 645321; 更改密码

alter user zhangsan account lock; 设置状态

alter user zhangsan account unlock;

alter user zhangsan default tablespace users 默认表空间
temporary tablespace TEMP; 临时表空间

drop user zhangsan;
```

用户权限的分配

单一与角色权限

角色:可以1个或者多个权限

系统权限:针对整个系统的大的权限

对象权限:针对某一个具体的对象上面的对象

登录权限: grant create session to 【username】 (系统权限)

表查询权限: grant 【权限: select、update、insert、delete】 on 【表】 scott.emp to 【username】 (使用system用户授予)

撤销权限: revoke [select、update、insert、delete] on [table] from [username]

```
SQL> create user cz identified by 123456; 创建用户
User created

SQL> conn cz/123456; 登录
SQL> grant create session to cz; 授予登录权限
Grant succeeded

SQL> conn cz/123456; 登录
Connected to Oracle Database 11g Enterprise Edition Release 11.2.0.1.0
Connected as cz

SQL> select * from scott.emp; 查询
select * from scott.emp
ORA-00942: 表或视图不存在

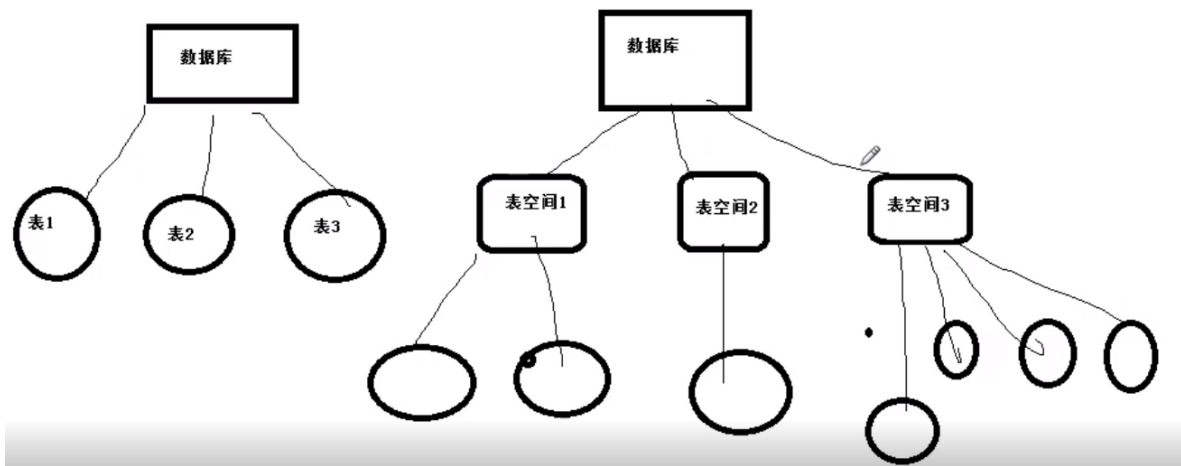
SQL> conn system/123456; 切换用户
Connected to Oracle Database 11g Enterprise Edition Release 11.2.0.1.0
Connected as system

SQL> grant select on scott.emp to 授予查询权限

SQL> revoke delete on scott.emp from cz; 撤销权限
Revoke succeeded
```

第3章Oracle 10g空间管理

创建，修改，删除，查询表空间



`select * from v$tablespace` 查看表空间

--查询表空间里边居然的文件

`select * from dba_data_files`

--查询表空间下面表的数量

`select count(*) from dba_tables where tablespace_name = 'MYTABLESPACE'`

/*

创建表空间：

Create tablespace 表空间名称；

DATAFILE 表空间存储的物理位置；

Size 表空间的大小；

create tablespace hy_tablespace

datafile 'c:/app/hp/hp.dbf'

size 10m

删除表空间：

Drop ablespace [表空间名称] (cascade级联删除表空间里边的表)

*/

面试题：

/*

1、创建名字为hy_tablespace的表空间，默认大小为10M

A.

create tablespace hy_tablespace

datafile 'c:/app/hp/hp.dbf'

size 10m

2、表空间文件的后缀名是？ xxx.DBF

3、哪一语句可实现增加表employee的列 NAME 的长度到 25 ？

ALTER TABLE employee MODIFY name VARCHAR2(25);

*/

SQL语言基础

查询: `select * from scott.tmp; select * from scott.emp where ename = 'WARD'`
插入: `insert into scott . emp (EMPNO, ENAME) values (9001, 'xj') ;`
数据回滚: `rollback;`
提交: `commit;`
更新: `update scott.emp set ename ='hs aj' where EMPNO = 9000;`
删除: `delete from scott.emp where EMPNO = 9000;`
多表查询: `select e.ename, d.dname from scott.emp e join scott.dept d on e.deptno = d.deptno;`
`select deptno, count (deptno) 员工数量from scott.emp grqup by deptno`

1、在Oracle中, 表VENDOR包含以下列:

VENDOR_ID NUMBER Primary Key
NAME VARCHAR2(30)
LOCATION_ID NUMBER
ORDER_DT DATE
ORDER_AMOUNT NUMBER(8,2)

下面对表VENDOR运用分组函数的子句合法的是 : `SELECT SUM(order_amount)`

2、在Oracle中, 表EMP包含以下列:

.....
NAME VARCHAR2(20)
ADDR VARCHAR2(60)
.....

要以NAME's address is ADDR格式返回数据, 以下SQL语句正确的是: `SELECT NAME || ' 's address is ' || ADDR FROM EMP`

3、带有 (FOR UPDATE) 字句的SELECT语句可以在表的一行或多行放置排他锁

4、哪一子句可实现 SELECT 语句查询单价小于5.00 的 ID_NUMBER: : : WHERE price < 5.00

5、部分匹配查询中有关通配符“_”的正确的叙述是 ()

- A. “_”代表多个字符
- B. “_”可以代表零个或多个字符
- C. “_”不能与“%”一同使用
- D. “_”代表一个字符

6、在Oracle中, 关于锁, 下列描述不正确的是:

- A. 锁用于在用户之间控制对数据的并发访问
- B. 可以将锁归类为行级锁和表级锁
- C. insert、update、delete语句自动获得行级锁
- //D. 同一时间只能有一个用户锁定一个特定的表

索引的增删改

```
/*  
创建索引:  
create index 【indexName】 on table(字段名称)  
删除索引:  
drop index 【索引名称】  
*/
```

视图的创建和删除

➤ 什么是 Oracle 视图

是一张虚拟的表

➤ Oracle 视图作用

1:安全性：隐藏字段、隐藏表名

2：可以做一个特定的查询，减少每次都要去写复杂的条件

创建视图语法：

```
create view 视图名称
```

```
as
```

```
sql语句
```

oracle 只读视图：

```
create or replace view 视图名称
```

```
as
```

```
sql语句 with read only
```

删除视图：

```
drop view 视图名称
```

修改视图：

```
create or replace ....
```

同义词和序列

类似于设置别名，通过别名来查询数据。

语法：create synonym 名称 for 表；

创建：create synonym ep for scott.emp；

查询：select ep；

删除：drop synonym ep；

第4章Oracle高级查询、事务、过程及函数

SQL函数

函数

必须有一个返回值

函数可以在sql语句中使用：

返回一个Int类型

语法：

```
Create function函数名称参数...
```

```
Return 类型--什么返回值
```

```
Is (as)
```

```
Begin
```

```
...处理逻辑
```

```
Return返回.
```

```

End;
/*
create or replace function func_cre
return int
as
begin
    return 1;
end;

select func_cre from dual;
*/

```

删除：
drop function 【函数名称】

```

create or replace function func_select (pname nvarchar2)
return int
as
    pasn nvarchar2;
begin
    select sal into pasn from secott.emp where ename = pname;
    return pasn;
end;
-----
select func_select('NSDDF') from dual;

```

多表查询

1、在Oracle中，以下不属于集合操作符的是（B）

A、UNION B、SUM C、MINUS D、INTERSECT

：解析

有时在实际应用中，为了合并多个SELECT语句的结果，可以使用集合操作符UNION、UNION ALL、INTERSECT、MINUS。这些操作符多用于数据量比较大的数据库，运行速度快，称为合并查询，也叫集合查询。

2、下列哪个集合操作符返回两个查询都有的行（D）

A.Union
B.Union all
C.Union only
D.Intersect

3、请选择能正确查询scott用户下部门表中部门下面有人的部门（C）

A.select d.dname from scott.emp e join scott.dept d on e.deptno = d.deptno
group by d.dname
B.select distinct d.dname from scott.emp e join scott.dept d on e.deptno =
d.deptno
C.select distinct d.dname from scott.emp e join scott.dept d on e.deptno =
d.deptno group by d.dname
D.select d.dname from scott.emp e join scott.dept d on e.deptno = d.deptno

事务处理

事务是一个整体要么一起执行要么都不执行，例如银行转账

特点： A:原子性，B:一致性，C:隔离性，D:持久性

commit 提交事务

rollback 数据回滚

delete scott. emp where empno = 1934; 通过rollback数据回滚，恢复被删除的数据，恢复到上一步数据

保存点：

存档：savepoint 保存点名称

回滚：rollback to保存点名称

```
/*
delete scott. emp where empno = 1934;
savepoint a;
delete scott. emp where empno = 9021;
rollback to a;
发现9021还在，但1934已经不在，这是回到了a保存点时的数据，
*/
show error: 查看报错
```

```
.
create or replace function fun_trans(e1 nvarchar2,e2 nvarchar2 ,money int)
return nvarchar2
as
prowcount int:=0;--申明一个存放受影响行数的临时变量
begin
    --把转出账户减去需要转账的金额
    update scott.emp set sal = sal-money where ename = e1;
    prowcount := prowcount+sql%rowcount;
    --把钱添加到收入的账户
    update scott.emp set sal = sal+money where ename = e2;
    prowcount := prowcount+sql%rowcount;

    if prowcount <2 then
        begin
            rollback;
            return '转账失败';
        end;
    else
        begin
            commit;
            return '转账成功';
        end;
    end if;
end;

begin
    dbms_output.put_line(fun_trans('SMITH','ALLEN',100));
end;
```

```
set serveroutput on;
```

1、在Oracle中，通过命令（C）可以释放锁

- A.INSERT
- B.DELETE
- C.ROLLBACK
- D.UNLOCK

2、在Oracle中，下面关于簇的描述不正确的是（B）

- A.簇是一组表，这些表拥有公共列，并且经常一起使用
- B.与每个表及其索引单独存储相比，簇能够提高INSERT语句的性能
- C.由于簇表中的所有行都使用相同的列作为公共主键，因此这些列仅为所有表存储一次，从而节省了存储空间
- D.经常更新的列不应选作簇键

3、哪个命令属于隐含提交命令（B）

- A.CREATE
- B.UPDATE
- C.COMMIT
- D.SELECT

4、在Oracle中，事务中使用下列SQL语句不会引起锁定（A）

- A.SELECT
- B.INSERT
- C.UPDATE
- D.DELETE

理解过程及函数

第5章Oracle PL/SQL编程基础

PL/SQL

PL/SQL语言基础

PL/SQL异常处理

```

declare
    i int:=1;
    p nvarchar2(64);
begin
    p:= 'hello';
    --模拟一个错误
    i:=i+p;
    dbms_output.put_line(i);
    exception
        when value_error then --数字或者值错误
            dbms_output.put_line('数字或者值错误');
        when others then --捕捉所有的异常
            dbms_output.put_line('程序发送错误');
end;

```

游标的使用

其实就是把查询的结果放入游标中,然后在去游标里边读取。相当于使用游标做了一个中介,而游标是可以作为参数进行传递的,所以游标可以在存储过程中起来数据集的桥梁作用

使用游标的一般步骤:

- 1、定义一个游标,也就是申明一块内容地址
注意游标只能使用 `is`, 不能使用 `as`
- 2、给游标关联一个 `sql` 语句
- 3、使用 `open` 打开游标,就会执行 `sql` 语句,把集合放入内存
- 4、循环使用 `fetch` 去读取游标的内容放入变量中
- 5、关闭游标,释放内存资源

```

declare cursor cursor_emp --1:定义游标
is
    select ename,sal from scott.emp where deptno = 20;--2: 给游标关联一个sql语句
    v_ename scott.emp.ename%type; --定义一个和emp表中ename字段一样的遍历
    v_sal scott.emp.sal%type; --定义一个和emp表中sal字段一样的遍历
begin
    open cursor_emp; --3: 打开游标
    loop
        fetch cursor_emp into v_ename,v_sal;--4:使用fetch去读取游标的内容放入变量中
        exit when cursor_emp%notfound; --退出条件: 当游标读取完内容时退出
        dbms_output.put_line(v_ename||':'||v_sal);
    end loop;
    close cursor_emp;--5、关闭游标, 释放内存资源
end;

```

```

declare cursor cursor_emp --1:定义游标
is
    select * from scott.emp where deptno = 20;--2: 给游标关联一个sql语句
    v_row scott.emp%rowtype; --定义一个和emp表中整行一样的数据类型
begin
    open cursor_emp; --3: 打开游标
    loop
        fetch cursor_emp into v_row;--4:使用fetch去读取游标的内容放入变量中
        exit when cursor_emp%notfound; --退出条件: 当游标读取完内容时退出
        dbms_output.put_line(v_row.ename||':'||v_row.sal);
    end loop;
    close cursor_emp;--5、关闭游标, 释放内存资源
end;

```

```

declare cursor cursor_emp --1:定义游标
is
    select * from scott.emp where deptno = 20;--2: 给游标关联一个sql语句
    v_row scott.emp%rowtype; --定义一个和emp表中整行一样的数据类型
begin
    open cursor_emp; --3: 打开游标
    loop
        fetch cursor_emp into v_row;--4:使用fetch去读取游标的内容放入变量中
        exit when cursor_emp%notfound; --退出条件: 当游标读取完内容时退出
        dbms_output.put_line(v_row.ename||':'||v_row.sal);
    end loop;
    close cursor_emp;--5、关闭游标, 释放内存资源
end;

```

```

select * from scott.emp where deptno = 20;

```

```

declare cursor cursor_emp --1:定义游标
is
    select * from scott.emp where deptno = 20;--2: 给游标关联一个sql语句
    v_row scott.emp%rowtype; --定义一个和emp表中行一样的数据类型
begin
    open cursor_emp; --3: 打开游标
    loop
        fetch cursor_emp into v_row;--4:使用fetch去读取游标的内容放入变量中
        exit when cursor_emp%notfound; --退出条件: 当游标读取完内容时退出
        dbms_output.put_line(v_row.ename||':'||v_row.sal||':'||v_row.job);
    end loop;
    close cursor_emp;--5、关闭游标, 释放内存资源
end;

```

```

declare cursor_emp(p_deptno number) --1:定义游标
is
    select * from scott.emp where deptno = p_deptno; --2: 给游标关联一个sql语句
    v_row scott.emp%rowtype; --定义一个和emp表中行一样的数据类型

begin
    open cursor_emp(30); --3: 打开游标
    loop
        fetch cursor_emp into v_row; --4:使用fecth去读取游标的内容放入变量中
        exit when cursor_emp%notfound; --退出条件: 当游标读取完内容时退出
        dbms_output.put_line(v_row.ename||':'||v_row.sal||':'||v_row.job);
    end loop;
    close cursor_emp; --5、关闭游标, 释放内存资源
end;

```

%FOUND 布尔型属性, 当最近一次读记录时成功返回, 则值为TRUE;

%NOTFOUND 布尔型属性, 与%FOUND相反;

%ISOPEN 布尔型属性, 当游标已打开时返回 TRUE;

%ROWCOUNT 数字型属性, 返回已从游标中读取的记录数

第6章Oracle PL/SQL高级特性

触发器的作用

触发器分类:

添加, 删除, 修改...

触发器的时间

触发是之前或者之后

Before

After

数据的同步:

历史记录

限制- 一些操作:

触发器的创建

触发器的触发级别:

例如表级触发, 行级触发等

```

create or replace trigger tri_ib_emp
before insert on scott.emp
begin
    --当emp表添加之前就会触发
    dbms_output.put_line('添加之前触发了|');
end;

```

```
SQL> insert into scott.emp(EMPNO,ENAME) values(9000,'cz');
```

添加之前触发了

```
1 row inserted
```

```
create or replace trigger tri_fb_emp
after insert on scott.emp
begin
    --当emp表添加之后就会触发
    dbms_output.put_line('添加之后触发了');
end;
```

```
SQL> insert into scott.emp(EMPNO,ENAME) values(9002,'cz');
```

添加之前触发了

添加之后触发了

```
1 row inserted
```

```
create or replace trigger tri_au_emp
after update on scott.emp
begin
    --当emp表添加之前就会触发
    dbms_output.put_line('更新之后触发了');
end;
```

```
SQL> update scott.emp set ENAME = 'aj' where EMPNO = 9002;
```

更新之后触发了

```
1 row updated
```

```
create or replace trigger tri_au_emp
after insert on scott.emp
for each row --行级触发器
begin
    dbms_output.put_line('你添加的数据是');
    dbms_output.put_line(:new.ename);
    dbms_output.put_line(:new.empno);
end;
```

```
SQL> insert into scott.emp(empno,ename) values(9100,'wtj');
```

添加之前触发了

你添加的数据是

```
wtj
```

```
9100
```

添加之后触发了

```
1 row inserted
```

```

create or replace trigger tri_au_emp
after update on scott.emp
for each row
begin
    dbms_output.put_line('更新之前的数据是');
    dbms_output.put_line(:old.ename);
    dbms_output.put_line('更新之后的数据是');
    dbms_output.put_line(:new.ename);
end;

```

```

SQL> update scott.emp set ename = 'ls' where empno = 9100;
更新之前触发了
更新之前的数据是
wtj
更新之后的数据是
ls
1 row updated

```

```

SQL> update scott.emp set ename = 'ls222222' where empno = 9100;
更新之前触发了
更新之前的数据是
ls
更新之后的数据是
ls222222
1 row updated

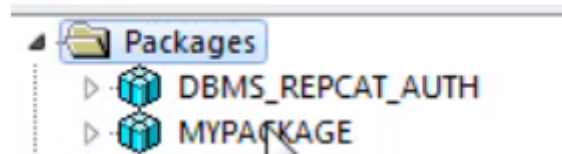
```

程序包的建立和使用

```

--创建包头
create or replace package mypackage
as
    function func_sum(a int ,b int) return int;
    procedure proc_show;
end mypackage;

```



--实现包体

```
create or replace package body mypackage
as
    function func_sum(a int ,b int) return int
    is
    begin
        return a+b;
    end;

    procedure proc_show
    as
    begin
        dbms_output.put_line('hello package');
    end;

end mypackage;

--
begin
    mypackage.proc_show();
end;

SQL> /
hello package
PL/SQL procedure successfully completed
```

第7章Oracle备份与恢复

常用备份与恢复

备份: exp userid=scott/123456@orc1 tables=(emp) file=E:\A.DMP

恢复: imp userid=scott/123456@orc1 file=E:\A.DMP

(两种方式: 1先删除表, 2命令)

full=y

ignora=y 忽略重复的表

imp userid=scott/123456@orc1 file=E:\A.DMP full=y ignora=y

其他用户备份: exp userid=system/123456@orc1 tables=(scott.emp) file=E:\emp.DMP

其他用户导入: imp userid=system/123456@orc1 file=E:\emp.DMP full=y

注意: 这个用户权限要高, 普通用户不行。需要dba权限。(grant dba to 【username】)

第8章Oracle优化技术

SQL语句优化

```
select * from scott.emp
```

```
select deptno,dname,loc from scott.emp :简单优化
```

```
-----  
select * from scott.emp where deptno = (select deptno from scott.dept where  
dname = ' ACCOUNTING' )  
or deptno = (select deptno from scott.dept where dname ='RESEARCH')  
优化1: select * from scott.emp where deptno in (select deptno from scott.dept  
where dname='ACCOUNT ING' or dname='RESEARCH')  
优化2: select * from scott.emp e where exists  
      (select deptno from scott.dept d where (dname='ACCOUNTING' or dname  
      ='RESEARCH') and e.deptno = d.deptno )
```

其他优化:

- 1、应尽量避免在where 子句中对字段进行null 值判断, 否则将导致引擎放弃使用索引而进行全表扫描。
- 2、应尽量避免在where子句中使用or来连接条件, 否则将导致引擎放弃使用索引而进行全表扫描, 如:union all
- 3、in和notin也要慎用, 否则会导致全表扫描, 如: 能用between 就不要用in了: (但是, 如果不是连续的话, 效率也不会太高)

```
create table t_stu(  
    stuid      number(10)    primary key,  
    stuname    varchar2(20) not null,  
    stusex     varchar2(2)   default '男' check(stusex in('男','女')),  
    address    varchar2(20)  
);
```

```
SQL> insert into t_stu(stuid,stuname) values(1,'aa')  
1 row inserted
```

```
SQL> insert into t_stu(stuid,stuname) values(2,'bb')  
1 row inserted
```

```
SQL> insert into t_stu(stuid,stuname,address) values  
1 row inserted
```

```
SQL> select * from t_stu;
```

STUID	STUNAME	STUSEX	ADDRESS
1	aa	男	
2	bb	男	
3	cc	男	沙坪坝

```
SQL> commit;
```

```
Commit complete
```

```
select * from t_stu where stuname ='aa' or stuname='bb'
```

```
select * from t_stu where stuname ='aa'  
union all  
select * from t_stu where stuname ='bb'  
  
select * from t_stu where stuid in (1,2,3)  
  
select * from t_stu where stuid between 1 and 3
```